

Dynamic programming (D.P.)

D.P is an optimization over Recursion

Recursion:

- ↳ we have a problem with fixed n
- ↳ we divide the problem into smaller problems

Ex: fact(5) $\Rightarrow 5 \times$ fact(4)

$4 \times$ fact(3)

$3 \times$ fact(2)

$2 \times$ fact(1)

Sub problem

↳ $(n-1), (n-2), (n-3)$

$$f(2) \Rightarrow 2$$

$$f(3) \Rightarrow 2$$

① Memoization

- Top-down approach
- Recursion

② Tabulation

- Bottom-up approach
- Iteration

→ Compute same subproblems repeatedly.

→ Compute subproblem only once ①

→ Store the result of subproblem ②

→ Reuse the result of subproblem solution again & again

$n \geq 0$ nth Fibonacci number

```
int fib(int n) {
```

```
    if (n == 0 || n == 1)
```

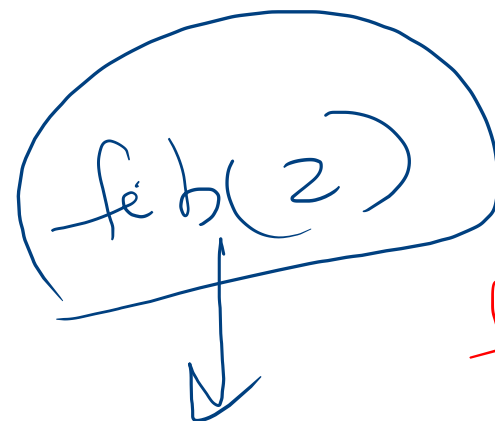
```
        return n;
```

```
    else
```

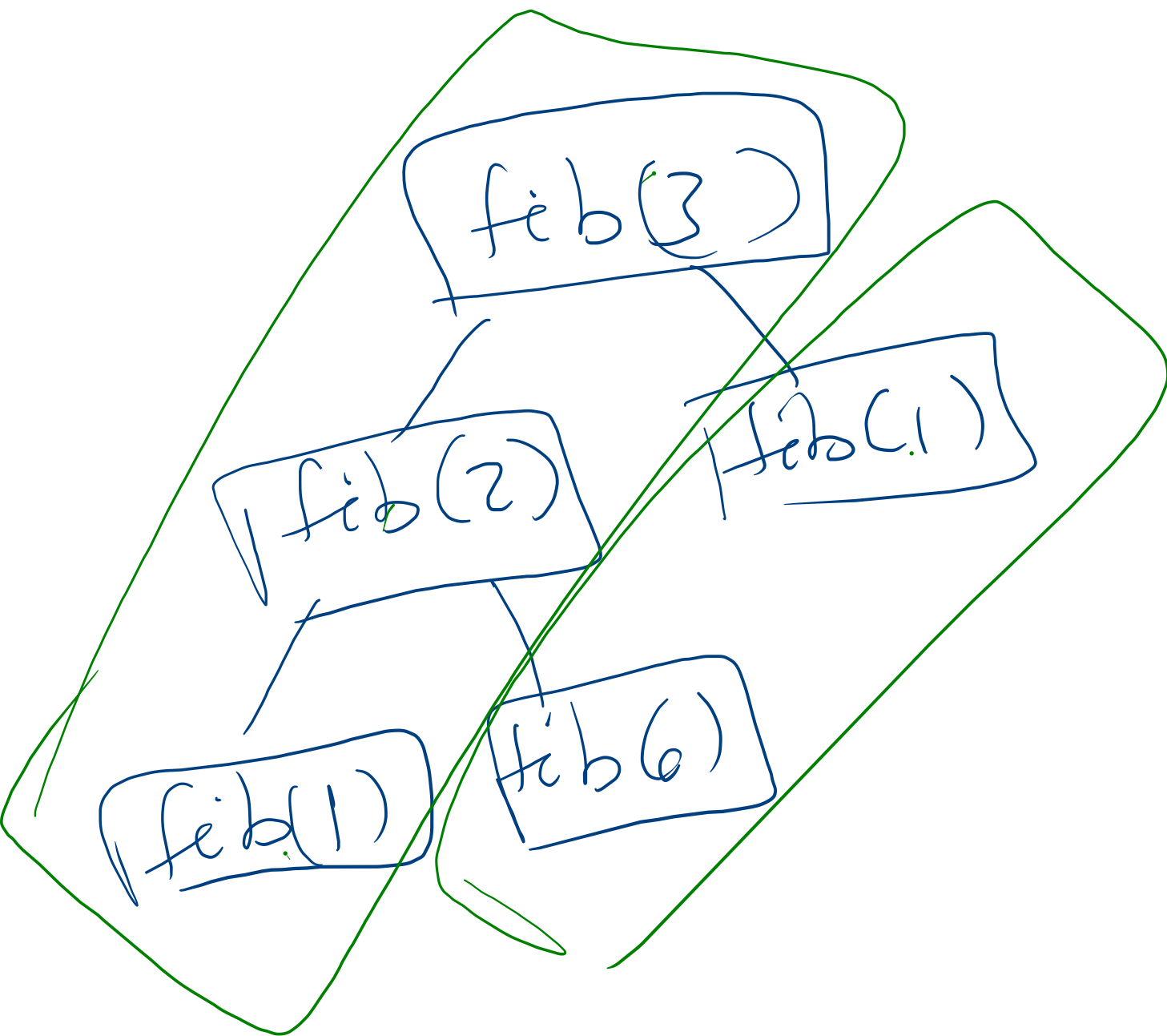
```
        return fib(n-1) + fib(n-2);
```

}

F_0	F_1	F_2	F_3	F_4	F_5	F_6
0	1	1	2	3	5	8
✓		1				



$$\frac{\text{fib}(2-1) + \text{fib}(2-2)}{\text{fib}(1) + \text{fib}(0)}$$



$$\cancel{n * (n-1)}$$

$$= 2n - 1$$

$$O(n)$$

Memorization

- ① Time — $O(n)$
- ~~②~~ Space — $O(n)$
- ③ Recursive
- ④ Easy to implement
- ⑤ Faster
- ⑥ More Space

Tabulation

- ① Time — $O(n)$
- ~~②~~ Space — $O(n)$
- ③ Iterative
- ④ Not Complex
- ⑤ very faster
- ⑥ low space

* Longest Common SubSequence

0 or more char
in same order
how they appn in str

Ex ① 'CDA' $n=3$

Subsequences $\rightarrow$ ", "C", "D", "A", "CD", "CA", "DA", "CDA" $\rightarrow 8$

Ex ② "DBA" $n=3$

Subseq $\rightarrow$ ", "D", "B", "A", "DB", "DA", "BA", "DBA" $\rightarrow 8$

Common $\Rightarrow$ ", "D", "A", "DA"

2

2^n

$2^3 = 8$

problem

① $S_1 = \text{"CDA"}$



$S_2 = \text{"DBA"}$



op = 2

find length of longest

Common Substring?

② $S_1 = \text{"gic"}$

$S_2 = \text{"gic"}$

op = 3

③ $S_1 = \text{"gic"}$

$S_2 = \text{"Sni"}$

o/p = 0



2 ← $\text{ LCS ("AXYZ", "BAZ")}$

$1 + \text{ LCS ("XY", "BA")}$

 LCS ("X", "BA")

 LCS ("XYZ", "B")

 LCS ("A", "BA")

 LCS ("AX", "B")

 LCS ("X", "B")

 LCS ("XYZ", "")

$1 + \text{ LCS ("", "B")}$

 LCS ("A", "B")

 LCS ("AX", "")

 LCS ("A", "")

 LCS ("", "B")

 LCS ("A", "B")

 LCS ("AX", "")

```

for(int i=1;i<=m;i++){
for(int j=1;j<=n;j++){
if (str1.charAt(i - 1) == str2.charAt(j - 1)) {
    table[i][j] = 1 + table[i-1][j-1];
} else {
    table[i][j] = Math.max(table[i-1][j],table[i][j-1]);
}
}
}

```

		0	1	2	3
0		0	0	0	0
1	A	0	0	1	1
2	X	0	0	1	1
3	Y	0	0	1	1
4	Z	0	0	1	2

$i=2$
 $j=1$
 $tab[m][n]$

$i=2, j=1$ → $tab[1][1]$ $tab[2][0]$
 $i=2, j=2$ → $tab[1][2]$ $tab[2][1]$
 $i=2, j=3$ → $tab[1][3]$ $tab[2][2]$

0 1 2 3
 0 1 2 3
 0 1 2 3

3 ← $\boxed{1cs(\text{"glc"}, \text{"glc"})}$

$1 + \boxed{1cs(\text{"glc"}, \text{"glc"})}$

2 $\boxed{1 + 1cs(\text{"glc"}, \text{"glc"})}$

1 $\boxed{1 + 1cs(\text{"glc"}, \text{"glc"})}$
0